

Assignment 3 – Simple Shell

Description :

This task implemented a simple Linux command-line interface using C. The program reads user input, parses the commands and their parameters, and executes these commands through fork and execvp. After the execution is completed, it reports the ID of the child process and its exit status.

This shell also supports pipeline functionality, where the output of one command becomes the input of the next command. This is achieved by creating multiple processes and connecting them with pipes. Special cases such as empty commands and exit commands are handled internally.

Approach:

I plan to divide the task into three stages for completion: input processing, command parsing, and process execution. Firstly, ensure that the basic commands can be executed correctly, and then expand the design to support pipeline functionality. This step-by-step approach helps to isolate problems during the development process. If a problem occurs, it can be resolved immediately before moving on to the next step.

The first stage is to safely collect the user input. I use the fgets function with a fixed buffer size to ensure that the input does not exceed the required limit. After read the line, I remove the newline character at the end to enable consistent string comparison and parsing. Then, I directly handle special cases in the shell, such as ignoring empty commands and recognizing the built-in "exit" command. Addressing these situations as early as possible can avoid unnecessary process creation and keep the control logic simple.

The second stage involves parsing the pipeline by splitting the input string at the pipe delimiter. Each split segment represents a separate command, and these commands must run in their respective processes. Before storing each segment, I will trim the leading whitespace characters to standardize the format. This includes checking for invalid pipeline structures, such as consecutive pipes which would result in an empty command segment. The parsed segments are stored in a fixed-size array, which limits the boundaries and simplifies the management of multiple pipeline commands.

For each parsed command segment, I convert it into a parameter array in the style of `argv` for `execvp`. I split the segment using spaces as the delimiter and dynamically expand the parameter list when a marker is found. To gradually expand the array as needed to determine the number of parameters, I use `realloc`. After collecting all the markers, I add a null terminator to meet the requirements of the `exec` interface.

The execution process is achieved by creating a child process for each command segment. If there is a pipe, a pipe will be dynamically created between adjacent commands, and there will be a variable to track the read end of the previous pipe so as to correctly connect multiple stages. In each child process, before executing the command through `execvp`, the standard input and/or output will be redirected according to its position in the pipe. The parent process will record the PID of each child process, close the unused file descriptors to avoid resource leakage, and wait for all child processes to complete to ensure proper synchronization and termination.

Finally, after all the child processes have been created, the parent process will use the `waitpid` function to synchronize with them. This can prevent the occurrence of zombie processes and enable the shell to report the termination results.

Issues and Resolutions:

1.

```
char **tokenize_command(char *cmd)
{
    int token_count = 0;

    // Remove newline character.
    if (cmd[strlen(cmd) - 1] == '\n')
    {
        cmd[strlen(cmd) - 1] = '\0';
    }
}
```

My first issue is that the newline characters were removed inside the "tokenize_command" function: This led to inconsistent behavior because the newline character handling was mixed with the tokenization logic. If multiple commands were processed, modifying the buffer at this point could interfere with the pipeline parsing.

I resolved it by the deletion of line breaks that are performed immediately after reading the input in the main function, rather than before the separation or word segmentation operations through pipes. This avoids unexpected situations during the command processing.

```
// Display prompt.
printf("%s", prompt);
char input[MAX_INPUT_SIZE];

// Check for EOF. Exit shell if reached.
if (!fgets(input, MAX_INPUT_SIZE, stdin))
{
    printf("EOF reached, exiting\n");
    exit(0);
}
```

2.

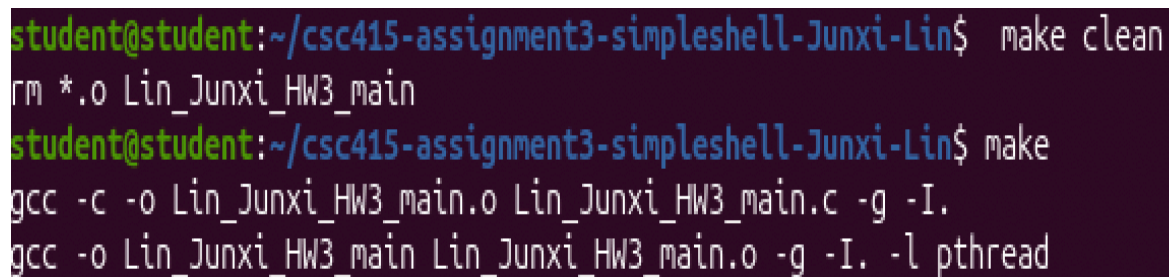
Issues:

The user presses Ctrl+D (normal EOF)

An error occurred during reading (I/O error)

Now, I use feof(stdin) to check if it is a normal EOF. Otherwise, I call perror to report the system error. This way, I can determine whether the termination was caused by the normal end-of-file character.

Screen shot of compilation:



```
student@student:~/csc415-assignment3-simpleshell-Junxi-Lin$ make clean
rm *.o Lin_Junxi_HW3_main
student@student:~/csc415-assignment3-simpleshell-Junxi-Lin$ make
gcc -c -o Lin_Junxi_HW3_main.o Lin_Junxi_HW3_main.c -g -I.
gcc -o Lin_Junxi_HW3_main Lin_Junxi_HW3_main.o -g -I. -l pthread
```

Screen shot(s) of the execution of the program:

```
enter command not found
student@student:~/csc415-assignment3-simpleshell-Junxi-Lin$ make run
./Lin_Junxi_HW3_main "Prompt> "
Prompt> ps
  PID TTY          TIME CMD
  4134 pts/0    00:00:00 bash
  4889 pts/0    00:00:00 make
  4890 pts/0    00:00:00 sh
  4891 pts/0    00:00:00 Lin_Junxi_HW3_m
  4892 pts/0    00:00:00 ps
Child 4892 finished with status: 0
Prompt> ps
  PID TTY          TIME CMD
  4134 pts/0    00:00:00 bash
  4889 pts/0    00:00:00 make
  4890 pts/0    00:00:00 sh
  4891 pts/0    00:00:00 Lin_Junxi_HW3_m
  4905 pts/0    00:00:00 ps
Child 4905 finished with status: 0
Prompt> ls foo
ls: cannot access 'foo': No such file or directory
Child 4906 finished with status: 2
Prompt> cat commands.txt | wc -l
7
Child 4909 finished with status: 0
Child 4910 finished with status: 0
Prompt> exit
student@student:~/csc415-assignment3-simpleshell-Junxi-Lin$ ps
  PID TTY          TIME CMD
  4134 pts/0    00:00:00 bash
  4912 pts/0    00:00:00 ps
student@student:~/csc415-assignment3-simpleshell-Junxi-Lin$ ls foo
ls: cannot access 'foo': No such file or directory
student@student:~/csc415-assignment3-simpleshell-Junxi-Lin$ cat commands.txt | wc -l
7
student@student:~/csc415-assignment3-simpleshell-Junxi-Lin$
```